GitLab vs Bitbucket
Plataformas en la nube para el versionamiento de software

Investigación comparativa y demostración práctica mediante dos proyectos equivalentes

Autor	**Leonardo Meza**
Programa	**Ingeniería de Sistemas**
Institución	**Universidad de la Costa (CUC)**
Tema	**Software de versionamiento en la nube**
Fecha	**24 de octubre de 2026**

Alcance del documento. Este informe describe qué son GitLab y Bitbucket, cómo funcionan internamente, qué características, planes y precios ofrecen, y presenta **dos demostraciones prácticas completas** (una por plataforma) construidas sobre un mismo proyecto de software, con el fin de evidenciar ventajas y desventajas de una frente a la otra. Todos los datos de precios y planes corresponden a la información publicada por los proveedores en octubre de 2026 y pueden variar según región, moneda e impuestos.

Índice de contenidos

1. Introducción, objetivos y metodología

2. Marco conceptual: versionamiento y plataformas en la nube

3. GitLab

3.1 Qué es · 3.2 Historia y licencia · 3.3 Arquitectura · 3.4 Cómo funciona · 3.5 Características · 3.6 Planes y precios · 3.7 Ventajas y desventajas

4. Bitbucket

4.1 Qué es · 4.2 Historia y licencia · 4.3 Arquitectura · 4.4 Cómo funciona · 4.5 Características · 4.6 Planes y precios · 4.7 Ventajas y desventajas

5. Comparación general de las dos plataformas

6. Ejemplo 1 — Demostración con GitLab

6.1 Descripción · 6.2 Estructura del proyecto · 6.3 El pipeline · 6.4 Flujo ejecutado · 6.5 Evidencias · 6.6 Análisis

7. Ejemplo 2 — Demostración con Bitbucket

7.1 Descripción · 7.2 Estructura del proyecto · 7.3 Los pipelines · 7.4 Flujo ejecutado · 7.5 Evidencias · 7.6 Análisis

8. Análisis comparativo de las dos demostraciones

9. Ventajas y desventajas de una plataforma sobre la otra

10. Conclusiones y recomendaciones

11. Referencias y fuentes consultadas

12. Anexos: archivos entregados y reproducción de la demostración

Cómo leer las demostraciones

Los capítulos 6 y 7 contienen las dos demostraciones. Los bloques de fondo oscuro corresponden a **salidas reales de terminal** obtenidas durante la ejecución; los bloques claros con borde de color corresponden a **archivos de configuración reales** incluidos en la entrega. Cuando se describe una acción que ocurre en la interfaz web de la plataforma (por ejemplo, aprobar un Merge Request), se indica expresamente que es una descripción del flujo y no una captura de pantalla.

1. Introducción, objetivos y metodología

1.1 Introducción

El control de versiones es la práctica que permite registrar, recuperar y auditar cada cambio realizado sobre el código fuente de un proyecto. Git es hoy el sistema de control de versiones distribuido dominante, pero Git por sí solo no resuelve dónde se aloja el repositorio, quién puede modificarlo, cómo se revisan los cambios ni cómo se prueban y despliegan automáticamente. De esa necesidad surgen las **plataformas de versionamiento en la nube**, entre las que destacan GitHub, GitLab y Bitbucket.

Este informe se concentra en las dos últimas. Aunque ambas alojan repositorios Git y ofrecen revisión de código y pipelines de integración continua, responden a filosofías muy distintas: GitLab se concibe como una **plataforma DevOps integral** que puede sustituir a varias herramientas y autoalojarse con paridad de funciones, mientras que Bitbucket es un **servicio de alojamiento de repositorios** profundamente integrado con el ecosistema de Atlassian (Jira, Confluence, Compass).

1.2 Objetivos

Objetivo general. Investigar y comparar las plataformas GitLab y Bitbucket, y demostrar mediante ejemplos prácticos sus ventajas y desventajas relativas.

Objetivos específicos.

- Definir qué es cada plataforma, su origen, su modelo de licenciamiento y su arquitectura de funcionamiento.
- Identificar las características principales de cada una, así como sus planes y precios vigentes.
- Construir un proyecto de software de prueba y ejecutarlo dos veces: una con el flujo de trabajo y el pipeline de GitLab y otra con el de Bitbucket, manteniendo constante el código de la aplicación.
- Documentar las salidas reales de las ejecuciones como evidencia.
- Contrastar los resultados y formular recomendaciones de elección según el contexto de uso.

1.3 Metodología

La investigación combinó tres actividades:

1. **Revisión documental y de fuentes web oficiales** (documentación técnica de GitLab y de Atlassian, páginas de precios, manual del producto y análisis de terceros de 2026), realizada el 24 de octubre de 2026. Las cifras de precios se contrastaron entre páginas oficiales, detectándose inconsistencias que se señalan en el capítulo correspondiente.
2. **Construcción de un proyecto de software común.** Se desarrolló *NotasApp*, un módulo Python de cálculo de notas académicas con validaciones, encapsulamiento y una batería de 16 pruebas unitarias ejecutables sin dependencias externas. El mismo código se utilizó en las dos demostraciones, de modo que cualquier diferencia observada provenga de la plataforma y no de la aplicación.
3. **Ejecución de dos flujos de trabajo completos** (uno por plataforma) sobre repositorios Git reales: rama corta de trabajo, confirmaciones, detección de un error por las pruebas automatizadas, corrección, fusión con *commit* de integración, etiquetado de versión y publicación en un servidor remoto. En cada caso se incluyó el archivo de configuración de CI/CD propio de la plataforma (`.gitlab-ci.yml` y `bitbucket-pipelines.yml`) y se verificó su validez sintáctica.

Limitación declarada de la demostración

El entorno de trabajo utilizado para elaborar este informe **no disponía de cuentas en GitLab.com ni en Bitbucket.org**. Por lo tanto, las demostraciones se ejecutaron sobre repositorios Git locales que *replican* fielmente el flujo de trabajo de cada plataforma (ramas, *commits* convencionales, fusiones con `--no-ff` equivalentes al Merge Request / Pull Request, etiquetas y *push* a un servidor remoto simulado). Los archivos de configuración de CI/CD son reales y están listos para ser utilizados en la plataforma: su sintaxis fue validada con un analizador YAML, y los pasos que el pipeline ejecutaría se corrieron localmente con los mismos comandos, obteniéndose las salidas que se muestran como evidencia. Las funciones que solo existen en la interfaz web (aprobaciones, tableros, informes visuales) se describen como parte del funcionamiento de la plataforma y se identifican como tales.

2. Marco conceptual: versionamiento y plataformas en la nube

2.1 Control de versiones y Git

Un **sistema de control de versiones (VCS)** registra la evolución de un conjunto de archivos a lo largo del tiempo. Git, creado por Linus Torvalds en 2005, es un VCS **distribuido**: cada copia del repositorio contiene el historial completo, y las operaciones de confirmación y consulta de historial se realizan localmente. La colaboración se logra sincronizando repositorios mediante *push* y *pull*.

Los conceptos que se usarán en las demostraciones son:

- **Repositorio:** base de datos de objetos que contiene el historial.
- **Commit:** instantánea inmutable del proyecto, identificada por un *hash* SHA-1/SHA-256, con autor, fecha y mensaje.
- **Rama (*branch*):** puntero móvil a un *commit*; permite líneas de trabajo paralelas.
- **Remoto:** copia del repositorio alojada en un servidor (GitLab.com, Bitbucket.org, etc.).
- **Etiqueta (*tag*):** marca fija de una versión, por ejemplo `v1.0.0`.

2.2 Qué aporta una plataforma en la nube

Git por sí solo no resuelve la colaboración a escala. Las plataformas en la nube añaden:

Capacidad	Descripción
Alojamiento remoto	Servidor siempre disponible con respaldo, control de acceso y alta disponibilidad.
Revisión de código	*Merge Requests* (GitLab) y *Pull Requests* (Bitbucket): propuesta de cambios con discusión en línea, aprobaciones y fusión controlada.
Protección de ramas	Reglas que impiden escribir directamente en ramas estables y exigen revisiones y comprobaciones previas.
CI/CD	Ejecución automática de compilación, pruebas y despliegue definida como código.
Trazabilidad	Historial auditable de quién cambió qué, cuándo y con qué resultado.
Gestión de trabajo	Incidencias, tableros y su vínculo con las confirmaciones de código.

2.3 Flujos de trabajo más comunes

Flujo	Idea central	Plataforma donde encaja mejor
Git Flow	Ramas de larga vida: `main`, `develop`, `release/*`, `hotfix/*`. Publicaciones planificadas.	Ambas, con más ceremonia
GitLab Flow	Ramas de entorno (`main` → `staging` → `production`); el código asciende por fusión entre ramas.	GitLab (nativo)
Ramas cortas por funcionalidad	Una rama por tarea que vive pocos días y se fusiona mediante MR/PR revisado. Es el flujo usado en este informe.	Ambas
Trunk-based	Integración muy frecuente en la rama principal, con pruebas automatizadas como red de seguridad (requiere buen CI).	GitLab (mejor soporte con *merge trains*)
Basado en *fork*	Cada contribuyente copia el repositorio y propone cambios al original. Común en proyectos abiertos.	Ambas

3. GitLab

3.1 Qué es GitLab

GitLab es una plataforma de DevOps que reúne en una sola aplicación el control de versiones Git, la integración y entrega continua (CI/CD), el registro de contenedores y paquetes, el análisis de seguridad, la gestión de incidencias y tableros, la documentación (*wiki*) y el alojamiento de sitios estáticos. Su propuesta de valor es **sustituir una cadena de herramientas por una única plataforma**, evitando integraciones externas y manteniendo una sola interfaz, un solo modelo de permisos y un solo historial de auditoría.

Aspecto	Detalle
Empresa	GitLab Inc. (empresa pública, cotiza en NASDAQ con el símbolo GTLB).
Origen	Proyecto iniciado en 2011 por Dmitriy Zaporozhets y Valery Sizov; la empresa se funda en 2014 con Sid Sijbrandij como cofundador.
Licencia	Modelo *open core*: **Community Edition (CE) bajo licencia MIT** y **Enterprise Edition (EE) bajo licencia propietaria** (código visible, uso restringido).
Formas de uso	GitLab.com (SaaS), **Self-Managed** (autoalojado con paridad funcional, incluso en redes aisladas) y GitLab Dedicated (SaaS de un solo inquilino).
Implementación	Ruby on Rails para la aplicación principal; Go para los componentes de alto rendimiento (Gitaly, Runner, Workhorse, Shell, Pages); Vue.js en el frontend; PostgreSQL y Redis como almacenes de datos.
Cuota de mercado SCM	Aproximadamente 16 % (frente al 68 % de GitHub y el 4 % de Bitbucket, según estimaciones de 2026).

3.2 Arquitectura y componentes

GitLab no es un único programa: es un conjunto de servicios que se despliegan juntos (paquete Omnibus, contenedor Docker o gráficos de Helm sobre Kubernetes). Los componentes principales son:

Componente	Función	Tecnología
Puma (Webservice)	Atiende las peticiones web y de API de la aplicación Rails.	Ruby
Workhorse	Proxy inteligente que descarga tráfico pesado (Git por HTTP, archivos, sondeo largo).	Go
Gitaly	Servicio RPC que centraliza el acceso de lectura y escritura a los repositorios Git. Presente en toda instalación.	Go
Gitaly Cluster	Réplica de Gitaly con *failover* automático y consistencia fuerte (componente Praefect).	Go
Sidekiq	Procesamiento de tareas en segundo plano (colas sobre Redis).	Ruby
PostgreSQL / Redis	Base de datos principal y almacén de caché, colas y mensajería.	—
GitLab Shell	Gestión del acceso Git por SSH y del control de permisos.	Go
GitLab Runner	Agente que ejecuta los *jobs* del pipeline. Binario único; se registra contra el servidor.	Go
Registries	Registro de imágenes de contenedor y de paquetes (npm, Maven, PyPI, NuGet, Go...).	—
GitLab Pages	Publicación de sitios estáticos desde el propio repositorio.	Go

El **Runner** merece atención porque es quien ejecuta el pipeline. Admite varios ejecutores: *shell* (rápido, sin aislamiento), *docker* (aislamiento por contenedor, el estándar), *kubernetes* (un pod por *job*, con escalado elástico), *docker-autoscaler* e *instance*. Un punto económicamente relevante: los *runners* autoalojados **no consumen los minutos de cómputo** de la suscripción.

3.3 Cómo funciona GitLab

El ciclo de trabajo típico sobre GitLab es el siguiente:

1. Rama
Se crea una rama corta desde `main`

→

2. Push
Se publica la rama en el remoto

→

3. Merge Request
Se abre el MR; el pipeline se dispara solo

→

4. Revisión
Aprobaciones, comentarios y reglas obligatorias

→

5. Fusión
Merge a `main` con el pipeline en verde

→

6. Despliegue
Entornos y etiqueta de versión

- **Merge Requests (MR).** Cada propuesta de cambio se discute en línea: comentarios por línea de código, sugerencias aplicables, hilos resueltos y aprobaciones. Se pueden exigir aprobaciones por rol o por propietario de código (`CODEOWNERS` , en Premium y Ultimate) y aplicar *merge trains* para serializar integraciones sin romper la rama principal.
- **Pipelines.** Se declaran en el archivo `.gitlab-ci.yml` de la raíz del repositorio. Se organizan en etapas (`stages`) y *jobs*. Incluyen funciones avanzadas como el grafo de dependencias `needs` (permite que un *job* no espere a toda su etapa), la inclusión de plantillas compartidas (`include` , `extends`), *pipelines* padre-hijo y *pipelines* multicitados entre proyectos.

- **Permisos y ramas protegidas.** Los roles de proyecto son Guest, Reporter, Developer, Maintainer y Owner. Las ramas protegidas definen quién puede hacer *push*, *force push* o fusionar.
- **Push options.** Desde la consola se puede pedir la creación del MR: `git push -o merge_request.create -o merge_request.target=main origin feature/login`. Es una capacidad exclusiva de GitLab entre las dos plataformas comparadas.

3.4 Características principales

Área	Características
Repositorios y revisión	Repositorios privados ilimitados, Merge Requests con aprobaciones, sugerencias aplicables, *merge trains*, ramas protegidas, búsqueda avanzada de código.
CI/CD	Pipelines en YAML con `stages`, `rules`, `needs`, caché, artefactos, entornos y despliegues manuales, revisión de aplicaciones (*Review Apps*), Auto DevOps.
Registros	Container Registry y Package Registry integrados por proyecto.
Seguridad (DevSecOps)	SAST, DAST, escaneo de dependencias, de contenedores, de secretos y de infraestructura como código, gestión de vulnerabilidades y cumplimiento normativo (suite completa en Ultimate).
Gestión del trabajo	Incidencias, tableros, etiquetas, hitos, épicas y hojas de ruta.
Documentación y publicación	Wiki integrada, GitLab Pages, entornos de despliegue con URL e historial de reversión.
Inteligencia artificial	GitLab Duo (asistente y agentes) basado en consumo de «GitLab Credits».
Integraciones	Jira (menos profunda que la de Bitbucket), Slack, Mattermost, Kubernetes, Terraform, nubes públicas.

3.5 Planes y precios

Característica	Free	Premium	Ultimate
Precio	0 USD	29 USD / usuario / mes (contrato anual)	Precio personalizado (se cotiza con el proveedor)
Usuarios	5 en el grupo de nivel superior	Ilimitados	Ilimitados
Minutos de cómputo al mes	400	10 000	50 000
Almacenamiento	10 GiB	500 GiB	500 GiB
Seguridad	SAST y detección de secretos limitados	Controles de revisión adicionales	Suite completa de seguridad y cumplimiento
Soporte	Comunidad	Soporte prioritario	Soporte prioritario y asesoría

Complementos: minutos adicionales a 10 USD por cada 1 000; almacenamiento adicional a 5 USD por 10 GiB al mes; créditos de GitLab Duo a 1 USD por crédito; planificación ágil empresarial a 15 USD por usuario al mes. En instalaciones autoalojadas (Self-Managed) el nivel Free no tiene el límite de 5 usuarios, y los *runners* propios no consumen minutos.

3.6 Ventajas y desventajas de GitLab

Ventajas

- **Plataforma única:** repositorios, CI/CD, registros, seguridad y gestión del trabajo en un solo lugar; menos herramientas que integrar.
- **CI/CD maduro:** dependencias entre *jobs* (paralelismo), inclusión de plantillas compartidas, *merge trains* y *pipelines* padre-hijo.
- **Autoalojamiento con paridad:** la versión autoalojada ofrece las mismas funciones que la nube, incluso en redes aisladas; clave para soberanía de datos.
- **Seguridad integrada:** análisis estático y dinámico, dependencias y contenedores sin herramientas de terceros.
- **Registros incluidos:** imágenes y paquetes por proyecto.
- **Código abierto (CE, MIT):** permite auditar y autoalojar gratuitamente.
- **Control de aprobaciones:** reglas por propietario de código y políticas obligatorias.

Desventajas

- **Curva de aprendizaje alta:** interfaz densa y muchos conceptos (etapas, entornos, reglas, agentes).
- **Coste:** el plan Premium (29 USD) es ocho veces más caro por usuario que el Standard de Bitbucket.
- **Consumo de recursos:** el autoalojado exige bastante memoria y mantenimiento especializado.
- **Nivel gratuito limitado en la nube:** 5 usuarios por grupo y solo 400 minutos de cómputo al mes.
- **Rendimiento percibido:** la interfaz puede resultar más pesada que la de plataformas más simples.
- **Integración con Jira menos profunda** que la nativa de Bitbucket.
- **Funciones de IA facturables por créditos**, con coste variable.

4. **Bitbucket**

4.1 Qué es Bitbucket

Bitbucket es el servicio de alojamiento de repositorios Git de Atlassian. Su diferencial no es competir en comunidad ni en amplitud de funciones, sino **conectar el código con el resto del ecosistema Atlassian**: las incidencias viven en Jira, la documentación en Confluence y el seguimiento de despliegues se refleja automáticamente en ambos. Está orientado a equipos corporativos que ya utilizan esas herramientas.

Aspecto	Detalle
Empresa	Atlassian (NASDAQ: TEAM).
Origen	Creado en 2008 por Jesper Nøhr, inicialmente centrado en Mercurial; adquirido por Atlassian en septiembre de 2010. El soporte de Mercurial finalizó el 1 de julio de 2020, por lo que hoy es exclusivamente Git.
Licencia	Propietaria. El servicio en la nube es cerrado; la versión autoalojada (Data Center) requiere licencia comercial.
Formas de uso	**Bitbucket Cloud** (SaaS gestionado por Atlassian) y **Bitbucket Data Center** (autoalojado, con clúster activo-activo).
Implementación	La nube es una plataforma SaaS cerrada de Atlassian (originalmente escrita en Python/Django y hoy sobre la plataforma Java de Atlassian); Data Center es un producto Java con clúster sobre Hazelcast.
Cuota de mercado SCM	Alrededor del 4 %.

4.2 Arquitectura y componentes

Componente	Función
Espacios de trabajo	Organización jerárquica *workspace* → proyectos → repositorios, con permisos en cada nivel.
Bitbucket Pipelines	Motor de CI/CD definido en `bitbucket-pipelines.yml`. Cada *step* se ejecuta en una imagen Docker.
Runners	Alojados por Atlassian (autoescalado) o autoalojados tras el cortafuegos. Se licencian por *slots* de compilación concurrentes.
Pipes	Componentes reutilizables del mercado de Atlassian para pasos frecuentes (despliegues, notificaciones, análisis).
Code Insights	Integración de resultados de herramientas externas (pruebas, cobertura, análisis de seguridad) dentro del Pull Request.
Entornos de despliegue	Seguimiento de despliegues por entorno, con permisos específicos en el plan Premium.
Atlassian Guard	Autenticación única y auditoría, con suscripción independiente.
Data Center	Clúster activo-activo con alta disponibilidad para instalaciones propias.

A diferencia de GitLab, Bitbucket **no incluye de forma nativa** registro completo de contenedores y paquetes, ni analizadores de seguridad propios: para esas funciones se recurre a Pipes de terceros (por ejemplo Snyk o SonarQube) o a servicios externos.

4.3 Cómo funciona Bitbucket

1. Rama
Se crea una rama corta desde `main`

→

2. Push
Se publica la rama en el remoto

→

3. Pull Request
El PR se crea desde la interfaz web

→

4. Merge checks
Mínimo de revisores y build correcto

→

5. Fusión
Merge a `main`

→

6. Despliegue
Steps por entorno con disparo manual

- **Pull Requests (PR).** Revisión en línea con comentarios, *tareas* pendientes, revisores por defecto y comprobaciones de fusión (*merge checks*). En el plan Free y Standard las comprobaciones son recomendaciones; en Premium pueden hacerse obligatorias.
- **Pipelines.** El archivo `bitbucket-pipelines.yml` define secciones `default`, `branches`, `pull-requests` y `tags`, cada una con una **lista secuencial de *steps***. Los *steps* reutilizables se declaran en `definitions` mediante anclas YAML (`&nombre` / `*nombre`). No existe dependencia selectiva entre *steps* ni inclusión de archivos externos.
- **Permisos.** Se definen a nivel de espacio de trabajo, proyecto, repositorio, rama y entorno, con restricciones de rama y revisores por defecto.
- **Autenticación.** Las contraseñas de cuenta y las *app passwords* fueron retiradas: desde el 28 de julio de 2026 el acceso por HTTPS exige **API tokens con permisos específicos** o *access tokens*; también se admite SSH.

Cambios disruptivos de Bitbucket en 2026

- **Issues y Wikis nativos retirados:** los datos se eliminaron a mediados de agosto de 2026; Atlassian dirige a Jira para las incidencias y a Confluence para la documentación. Bitbucket deja de ser una solución autosuficiente de planificación.
- **App passwords eliminados (28 de julio de 2026):** toda automatización que usaba contraseñas de aplicación debe migrarse a API tokens con permisos.

4.4 Características principales

Área	Características
Repositorios y revisión	Repositorios privados ilimitados, Pull Requests con tareas y revisores por defecto, restricciones de rama, comprobaciones de fusión, descripciones de PR generadas con inteligencia artificial.
CI/CD	Bitbucket Pipelines con secciones por evento, *steps* reutilizables, cachés, artefactos, entornos de despliegue y disparos manuales.
Extensibilidad	Más de un centenar de aplicaciones conectadas y Pipes del mercado (Snyk, SonarQube, despliegues a AWS/Azure…).
Integración Atlassian	Jira (enlaces automáticos entre commits y tareas, «smart commits», transiciones de estado), Confluence, Trello, Compass y el seguimiento de despliegues.
Seguridad	Sin análisis propio; listas de IP permitidas, doble factor obligatorio, commits firmados y permisos de despliegue en Premium, más Atlassian Guard.
Registros	Bitbucket Packages, más limitado que el de GitLab.
Sitios estáticos	No ofrece un servicio equivalente a GitLab Pages.

4.5 Planes y precios

Característica	Free	Standard	Premium
Precio	0 USD	3,65 USD / usuario / mes	7,25 USD / usuario / mes
Usuarios	Hasta 5	Ilimitados	Ilimitados
Minutos de compilación al mes	50	2 500	3 500
Almacenamiento de archivos grandes (LFS)	1 GB	5 GB	10 GB
Comprobaciones de fusión	Recomendadas	Recomendadas	**Obligatorias**
Seguridad de red	—	—	Listas de IP y doble factor obligatorio
Code Insights	3 integraciones	3 integraciones	Ilimitadas
Soporte	Comunidad	Estándar	Premium con acuerdo de nivel de servicio

Complementos: minutos adicionales a 10 USD por cada 1 000 al mes; 100 GB adicionales de LFS por 10 USD al mes; almacenamiento excedente a 0,35 USD por GB al mes. Nota de verificación: durante la investigación la página `bitbucket.org/product/pricing` mostró 3,65 / 7,25 USD por usuario, mientras que `atlassian.com/software/bitbucket/pricing` llegó a mostrar 8,00 USD para Premium y varios análisis citaban 3,00 / 6,00 USD. Se recomienda confirmar el valor vigente antes de presupuestar.

4.6 Ventajas y desventajas de Bitbucket

Ventajas

- **Integración nativa con Jira y Confluence:** enlaces automáticos, cambios de estado y seguimiento de despliegues.
- **Precio competitivo:** 3,65 USD por usuario en Standard, el más bajo de los planes de pago analizados.
- **Curva de aprendizaje baja:** configuración simple y revisión muy directa.
- **Nivel gratuito útil:** repositorios privados ilimitados para equipos de hasta 5 personas.
- **DATA Center con clúster activo-activo** para alta disponibilidad autoalojada.
- **Controles empresariales accesibles** en Premium: IP permitidas, doble factor obligatorio, comprobaciones de fusión forzosas.
- **Ecosistema de Pipes** que cubre funciones no nativas.

Desventajas

- **CI/CD menos potente:** sin dependencias selectivas, sin plantillas compartidas y sin *merge trains*; ejecución estrictamente secuencial.
- **Sin seguridad nativa:** depende de herramientas externas.
- **Sin registro completo** de contenedores y paquetes.
- **Dependencia del ecosistema Atlassian:** sin Jira y Confluence la plataforma pierde buena parte de su valor.
- **Retiro de Issues y Wikis (2026):** obliga a migrar y a pagar otras herramientas.
- **Nivel gratuito muy limitado en CI/CD:** solo 50 minutos de compilación al mes.
- **Menor paridad** entre la nube y la versión autoalojada.
- **Sin alojamiento de sitios estáticos** ni funciones de IA tan amplias como las de la competencia.

5. Comparación general de las dos plataformas

Criterio	GitLab	Bitbucket
Enfoque	Plataforma DevOps integral	Alojamiento Git integrado con Atlassian
Empresa / licencia	GitLab Inc. — *open core* (CE MIT, EE propietaria)	Atlassian — propietaria
Precio de entrada de pago	29 USD / usuario / mes (Premium)	3,65 USD / usuario / mes (Standard)
Nivel gratuito	5 usuarios, 400 minutos de cómputo, 10 GiB	5 usuarios, 50 minutos de compilación, 1 GB de LFS
CI/CD	Muy completo: etapas, dependencias, plantillas, *merge trains*	Sencillo: listas de *steps* por tipo de evento
Registros de contenedores y paquetes	Integrados y completos	Limitados
Seguridad integrada	Sí (suite completa en Ultimate)	No nativa; mediante Pipes de terceros
Integraciones	Amplias pero genéricas; Jira menos profunda	Excelente dentro de Atlassian; limitada fuera
Autoalojado	Paridad funcional total, incluso en red aislada	Data Center con clúster, menor paridad
Gestión del trabajo	Incidencias, tableros y épicas incluidos	Depende de Jira (Issues y Wikis retirados en 2026)
Facilidad de uso	Curva alta, interfaz densa	Curva baja, interfaz simple
Escalabilidad	Muy alta (Gitaly Cluster, híbrido nativo en la nube)	Buena en la nube; clúster en Data Center
Sitios estáticos	GitLab Pages	No disponible
Autenticación por HTTPS	Personal Access Token o SSH	API token obligatorio (app passwords retirados en julio de 2026)
Cuota de mercado SCM	≈ 16 %	≈ 4 %

Lectura de la tabla

No existe una plataforma «mejor» en abstracto: GitLab maximiza **capacidad y control** a cambio de coste y complejidad, mientras que Bitbucket maximiza **simplicidad y economía** a cambio de depender del ecosistema Atlassian y de complementar su CI/CD con herramientas externas.

6. Ejemplo 1 — Demostración con GitLab

6.1 Descripción del ejemplo

Se construyó el proyecto **NotasApp**: un módulo Python para el cálculo de notas académicas con validación de rangos, promedio aritmético, promedio ponderado y decisión de aprobación. El proyecto no requiere dependencias externas (solo la biblioteca estándar), de modo que puede ejecutarse en cualquier equipo o contenedor, y cuenta con **16 pruebas unitarias**.

Sobre ese proyecto se aplicó el flujo de trabajo de **GitLab Flow con ramas cortas por funcionalidad y Merge Requests**, junto con un pipeline declarado en `.gitlab-ci.yml` con tres etapas: `build` → `test` → `deploy`.

> **Objetivo del ejemplo**
>
> Demostrar, con evidencia real, (a) que el pipeline detecta automáticamente un error antes de integrar el cambio, (b) que el despliegue queda restringido a la rama principal y a una aprobación manual, y (c) qué nivel de complejidad exige la configuración de GitLab para lograr ese control.

6.2 Estructura del proyecto

```
demos/gitlab/
├── .gitignore                # exclusión de artefactos de Python
├── .gitlab-ci.yml            # pipeline CI/CD de GitLab (76 líneas)
├── README.md                 # descripción y flujo de trabajo
├── requirements.txt          # sin dependencias externas
├── scripts/
│   └── deploy.py             # despliegue simulado invocado por el pipeline
├── src/
│   └── notas.py              # lógica de negocio (validación, promedio, ponderado)
└── tests/
    └── test_notas.py         # 16 pruebas unitarias con unittest
```

El módulo principal

```
def promedio_ponderado(notas, pesos):
    """Calcula el promedio ponderado de las notas según sus pesos."""
    if len(notas) != len(pesos):
        raise NotaInvalidaError("La cantidad de notas y de pesos debe coincidir")
    if sum(pesos) <= 0:
        raise NotaInvalidaError("La suma de los pesos debe ser mayor que cero")
    total = sum(validar_nota(n) * p for n, p in zip(notas, pesos))
    return round(total / sum(pesos), 2)
```

Fragmento final de `src/notas.py`. Las dos validaciones y el redondeo fueron añadidos durante la demostración a raíz de un fallo detectado por las pruebas.

6.3 El pipeline de GitLab

```yaml
# .gitlab-ci.yml - Pipeline CI/CD de GitLab para NotasApp (demostración)

stages:
  - build
  - test
  - deploy

default:
  image: python:3.12-slim

variables:
  PIP_CACHE_DIR: "$CI_PROJECT_DIR/.pip-cache"

# Caché compartida entre jobs, invalidada cuando cambia requirements.txt
cache:
  key:
    files:
      - requirements.txt
  paths:
    - .pip-cache/

# ------------------------------ BUILD ------------------------------
build:
  stage: build
  script:
    - python -m pip install --cache-dir .pip-cache -r requirements.txt
    - python -m compileall -q src/
  artifacts:
    name: "build-$CI_COMMIT_SHORT_SHA"
    paths:
      - src/
    expire_in: 1 week

# ------------------------------- TEST ------------------------------
test:unitarias:
  stage: test
  needs: ["build"]              # DAG: arranca sin esperar otros jobs de 'test'
  script:
    - python -m unittest discover -s tests -v
  artifacts:
    when: always
    paths:
      - tests/
    expire_in: 3 days

test:estilo:
  stage: test
  script:
    - python -m pip install ruff
    - ruff check src/

# ------------------------------ DEPLOY -----------------------------
deploy:staging:
  stage: deploy
  needs: ["test:unitarias", "test:estilo"]
  environment:
    name: staging
    url: https://staging.notasapp.example.com
  script:
    - python scripts/deploy.py staging
  rules:
    - if: '$CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH'

deploy:produccion:
  stage: deploy
  needs: ["test:unitarias"]
  environment:
    name: produccion
    url: https://notasapp.example.com
  script:
    - python scripts/deploy.py produccion
  rules:
    - if: '$CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH'
      when: manual              # requiere aprobación manual en la interfaz
    - when: never
```

Verificación de la sintaxis

El archivo fue analizado con un intérprete YAML para comprobar su validez antes de la entrega:

```
ARCHIVO YAML VALIDO: demos/gitlab/.gitlab-ci.yml
  claves de nivel superior: ['stages', 'default', 'variables', 'cache',
                             'build', 'test:unitarias', 'test:estilo',
                             'deploy:staging', 'deploy:produccion']
  etapas: ['build', 'test', 'deploy']
  jobs definidos: ['build', 'test:unitarias', 'test:estilo',
                   'deploy:staging', 'deploy:produccion'] | total: 5
```

6.4 Flujo de trabajo ejecutado

Estos son los comandos que se ejecutaron realmente sobre el repositorio de demostración:

```
# 1) Inicialización del repositorio y versión base de NotasApp
git init -b main
git add -A
git commit -m "feat: version inicial de NotasApp con validacion, promedio y aprobacion"

# 2) Rama corta de funcionalidad (equivale a lo que un Merge Request integrará)
git switch -c feature/promedio-ponderado
#   ... se implementa promedio_ponderado y sus 4 pruebas ...

# 3) Confirmación del cambio
git commit -m "feat: agrega el calculo de promedio ponderado"

# 4) Ejecución de las pruebas (lo que haría el job test:unitarias)
python -m unittest discover -s tests        # -> FALLA (1 fallo y 1 error)

# 5) Corrección del código
git commit -m "fix: valida la suma de pesos y redondea el promedio ponderado"
python -m unittest discover -s tests        # -> 16 pruebas OK

# 6) Higiene del repositorio: se dejan de versionar los artefactos generados
git rm -r --cached src/__pycache__ tests/__pycache__
git commit -m "chore: excluye los artefactos de Python del control de versiones"

# 7) Fusión controlada: equivale al botón "Merge" del Merge Request !1
git switch main
git merge --no-ff feature/promedio-ponderado \
    -m "Merge branch 'feature/promedio-ponderado' into 'main'"

# 8) Etiqueta de versión
git tag -a v1.0.0 -m "Release 1.0.0"

# 9) Segunda iteración: rama de corrección (equivalente al Merge Request !2)
git switch -c fix/script-despliegue-portable
git commit -m "fix(ci): migra el despliegue a un script portable en Python"
git switch main
git merge --no-ff fix/script-despliegue-portable \
    -m "Merge branch 'fix/script-despliegue-portable' into 'main'"

# 10) Publicación en el servidor remoto (GitLab.com en un caso real)
git push -u origin --all
git push origin --tags
```

En la plataforma real, el paso 7 se realiza con el botón *Merge* del Merge Request, después de que el pipeline esté en verde y de cumplirse las aprobaciones exigidas. GitLab permite además crear el MR desde la propia consola con `git push -o merge_request.create -o merge_request.target=main origin feature/promedio-ponderado`.

6.5 Evidencias de ejecución

Evidencia 1 — El pipeline detecta el error

Tras confirmar la nueva funcionalidad se ejecutaron las pruebas, obteniéndose un fallo y un error. Este es el comportamiento que en la plataforma bloquearía el Merge Request:

```
=== GITLAB :: job test:unitarias (commit con error) ===
........F.E.....
======================================================================
ERROR: test_rechaza_suma_de_pesos_en_cero
----------------------------------------------------------------------
Traceback (most recent call last):
  File ".../demos/gitlab/tests/test_notas.py", line 79, in test_rechaza_suma_de_pesos_en_cero
    promedio_ponderado([3.0], [0])
  File ".../demos/gitlab/src/notas.py", line 70, in promedio_ponderado
    return total / sum(pesos)
ZeroDivisionError: float division by zero

======================================================================
FAIL: test_promedio_ponderado_redondea_a_dos_decimales
----------------------------------------------------------------------
AssertionError: 3.466666666666667 != 3.47

----------------------------------------------------------------------
Ran 16 tests in 0.008s

FAILED (failures=1, errors=1)
```

Salida real de la ejecución de las pruebas sobre el código con el defecto.

Qué demuestra esta evidencia

El código llegó a tener dos defectos reales: una división por cero cuando la suma de pesos es nula y la ausencia del redondeo prometido en la documentación. Las pruebas automatizadas los detectaron **antes** de integrar el cambio a la rama principal. En GitLab, ese mismo resultado aparecería como pipeline fallido dentro del Merge Request, con el informe de pruebas adjunto, y con las reglas de aprobación activas el botón de fusión quedaría deshabilitado.

Evidencia 2 — Corrección y pipeline en verde

```
=== GITLAB :: job test:unitarias (tras la correccion) ===
................
----------------------------------------------------------------------
Ran 16 tests in 0.001s

OK
```

Las 16 pruebas pasan después de añadir las validaciones y el redondeo.

Evidencia 3 — Restricción del despliegue

El job de despliegue solo se ejecuta sobre la rama principal y, en producción, exige aprobación manual (`when: manual`). Así se comportó el paso de despliegue al ejecutarse sobre `main`:

```
=== GITLAB :: job deploy:produccion ===
== Despliegue de NotasApp ==
Entorno .....: produccion
Commit ......: 07a74e1b
Rama ........: main
Fecha .......: 2026-10-24 10:12:27
Resultado ...: despliegue en produccion completado.
```

El job `deploy:produccion` se activa únicamente sobre la rama por defecto, gracias a la regla `$CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH`, y requiere una acción manual adicional.

Evidencia 4 — Historial resultante y etiqueta de versión

```
=== GITLAB :: historial final ===
*   07a74e1 (HEAD -> main) Merge branch 'fix/script-despliegue-portable' into 'main'
|\
| * 90f8e46 (fix/script-despliegue-portable) fix(ci): migra el despliegue a un script portable en Python
|/
*   a89cf2a (tag: v1.0.0) Merge branch 'feature/promedio-ponderado' into 'main'
|\
| * 3248d35 (feature/promedio-ponderado) chore: excluye los artefactos de Python del control de versiones
| * 7a30b05 fix: valida la suma de pesos y redondea el promedio ponderado
| * 6e6fd95 feat: agrega el calculo de promedio ponderado
|/
* 8d1ff8a feat: version inicial de NotasApp con validacion, promedio y aprobacion

=== GITLAB :: estado del repositorio ===
## main
=== GITLAB :: pruebas sobre main ===
Ran 16 tests in 0.001s

OK

=== GITLAB :: etiquetas ===
v1.0.0 -> Release 1.0.0
```

Cada fusión conserva su *commit* de integración (`--no-ff`), lo que hace trazable qué cambio entró, cuándo y desde qué rama; el árbol queda limpio y etiquetado.

Evidencia 5 — Publicación en el remoto

```
=== GITLAB :: git push -u origin --all ===
branch 'feature/promedio-ponderado' set up to track 'origin/feature/promedio-ponderado'.
branch 'fix/script-despliegue-portable' set up to track 'origin/fix/script-despliegue-portable'.
branch 'main' set up to track 'origin/main'.
To .../gitlab-notasapp.git
 * [new branch]      feature/promedio-ponderado -> feature/promedio-ponderado
 * [new branch]      fix/script-despliegue-portable -> fix/script-despliegue-portable
 * [new branch]      main -> main
=== GITLAB :: git push origin --tags ===
 * [new tag]         v1.0.0 -> v1.0.0
```

Se publicaron las tres ramas y la etiqueta. Sobre un servidor GitLab real, el primer *push* de una rama nueva habilita el enlace para crear el Merge Request.

6.6 Análisis del ejemplo de GitLab

Lo que el ejemplo demuestra a favor

- **Control fino del flujo:** la palabra clave `needs` permite que `test:unitarias` arranque sin esperar a `test:estilo`, algo imposible en la lista secuencial de Bitbucket.
- **Reglas expresivas:** `rules` con comparación de variables evita desplegar desde una rama de trabajo.
- **Entornos con identidad:** `environment` define nombre y URL, y la plataforma conserva el historial de despliegues y permite revertir.
- **Artefactos y caducidad:** `artifacts` con `expire_in` y `when: always` garantiza que los resultados se conserven incluso cuando el *job* falla, justo lo necesario para diagnosticar el error del ejemplo.
- **Caché inteligente:** la clave derivada de `requirements.txt` invalida la caché solo cuando cambian las dependencias.
- **Creación del MR desde la consola:** reduce pasos manuales en el flujo diario.

Lo que el ejemplo evidencia en contra

- **Densidad de configuración:** 76 líneas y nueve claves de nivel superior para un proyecto mínimo. Esa potencia se paga en tiempo de aprendizaje.
- **Dependencia de la interfaz para cerrar el ciclo:** la aprobación manual y las reglas de aprobación se gestionan en la web, lo que añade un paso humano.
- **Mayor consumo de recursos:** el *job* de estilo descarga herramientas en cada ejecución si no se cachean; optimizar el pipeline exige conocimiento adicional.
- **Coste:** en la nube, cada minuto consumido cuenta sobre los 400 del plan gratuito, salvo que se usen *runners* propios.
- **Ruido por artefactos:** el propio ejemplo mostró que sin un `.gitignore` cuidado se pueden versionar archivos generados como `__pycache__`.

7. Ejemplo 2 — Demostración con Bitbucket

7.1 Descripción del ejemplo

Se utilizó **exactamente el mismo código** de NotasApp (mismas 16 pruebas, misma lógica) en un segundo repositorio, ahora con el flujo de **ramas cortas y Pull Request** propio de Bitbucket y con los pipelines declarados en `bitbucket-pipelines.yml`. El objetivo es que la comparación sea justa: lo único que cambia es la plataforma.

7.2 Estructura del proyecto

```
demos/bitbucket/
├── .gitignore
├── bitbucket-pipelines.yml    # CI/CD de Bitbucket (67 líneas)
├── README.md
├── requirements.txt
├── scripts/
│   └── deploy.py
├── src/
│   └── notas.py               # idéntico al del ejemplo de GitLab
└── tests/
    └── test_notas.py          # las mismas 16 pruebas
```

7.3 Los pipelines de Bitbucket

```
# bitbucket-pipelines.yml - Pipeline CI/CD de Bitbucket para NotasApp

image: python:3.12-slim

definitions:
  caches:
    pip-cache: /root/.cache/pip        # caché de dependencias reutilizable

  steps:                               # steps reutilizables mediante anclas YAML (& / *)
    - step: &build
        name: Compilar
        caches:
          - pip-cache
        script:
          - python -m pip install --cache-dir /root/.cache/pip -r requirements.txt
          - python -m compileall -q src/
        artifacts:
          - src/**

    - step: &test
        name: Pruebas unitarias
        caches:
          - pip-cache
        script:
          - python -m unittest discover -s tests -v
        artifacts:
          - tests/**

    - step: &estilo
        name: Revisión de estilo
        caches:
          - pip-cache
        script:
          - python -m pip install ruff
          - ruff check src/

pipelines:
  # Se ejecuta en cualquier push que no coincida con una regla más específica
  default:
    - step: *build
    - step: *test

  # Pull Requests: además del build y las pruebas, exige revisión de estilo
  pull-requests:
    '**':
      - step: *build
      - step: *test
      - step: *estilo

  # Rama principal: incluye despliegues a los entornos
  branches:
    main:
      - step: *build
      - step: *test
      - step: *estilo
      - step:
          name: Desplegar en Staging
          deployment: staging
          script:
            - python scripts/deploy.py staging
      - step:
          name: Desplegar en Producción
          deployment: production
          trigger: manual            # requiere aprobación manual en la interfaz
          script:
            - python scripts/deploy.py production
```

Verificación de la sintaxis

```
ARCHIVO YAML VALIDO: demos/bitbucket/bitbucket-pipelines.yml
  claves de nivel superior: ['image', 'definitions', 'pipelines']
  claves de pipelines: ['default', 'pull-requests', 'branches']
  steps reutilizables: ['Compilar', 'Pruebas unitarias', 'Revisión de estilo']
```

7.4 Flujo de trabajo ejecutado

```
# 1) Versión base
git init -b main
git add -A
git commit -m "feat: version inicial de NotasApp con validacion, promedio y aprobacion"

# 2) Rama de funcionalidad (será el origen del Pull Request #1)
git switch -c feature/promedio-ponderado
git commit -m "feat: agrega el calculo de promedio ponderado"
python -m unittest discover -s tests        # -> FALLA (1 fallo y 1 error)
git commit -m "fix: valida la suma de pesos y redondea el promedio ponderado"
python -m unittest discover -s tests        # -> 16 pruebas OK
git rm -r --cached src/__pycache__ tests/__pycache__
git commit -m "chore: excluye los artefactos de Python del control de versiones"

# 3) Fusión: en Bitbucket el PR se aprueba en la interfaz web y deja este mensaje
git switch main
git merge --no-ff feature/promedio-ponderado \
    -m "Merged in feature/promedio-ponderado (pull request #1)"
git tag -a v1.0.0 -m "Release 1.0.0"

# 4) Pull Request #2
git switch -c fix/script-despliegue-portable
git commit -m "fix(ci): migra el despliegue a un script portable en Python"
git switch main
git merge --no-ff fix/script-despliegue-portable \
    -m "Merged in fix/script-despliegue-portable (pull request #2)"

# 5) Publicación
git push -u origin --all
git push origin --tags
```

Diferencia operativa importante: en Bitbucket **no existe una opción de *push* que cree el Pull Request**; la propuesta se abre desde la interfaz web o mediante la API REST. Además, el flujo real se completó con las comprobaciones de fusión (*merge checks*) configuradas para exigir pipeline correcto y un número mínimo de revisores; en los planes Free y Standard estas comprobaciones se muestran como recomendaciones y solo en Premium pueden hacerse obligatorias.

7.5 Evidencias de ejecución

Evidencia 1 — El mismo defecto, detectado por las pruebas

```
=== BITBUCKET :: step Pruebas unitarias (commit con error) ===
........F.E.....
======================================================================
ERROR: test_rechaza_suma_de_pesos_en_cero
----------------------------------------------------------------------
Traceback (most recent call last):
  File ".../demos/bitbucket/tests/test_notas.py", line 79, in test_rechaza_suma_de_pesos_en_cero
    promedio_ponderado([3.0], [0])
  File ".../demos/bitbucket/src/notas.py", line 70, in promedio_ponderado
    return total / sum(pesos)
ZeroDivisionError: float division by zero

======================================================================
FAIL: test_promedio_ponderado_redondea_a_dos_decimales
----------------------------------------------------------------------
AssertionError: 3.466666666666667 != 3.47

----------------------------------------------------------------------
Ran 16 tests in 0.008s

FAILED (failures=1, errors=1)
```

En Bitbucket este resultado haría fallar el *step* «Pruebas unitarias» y el Pull Request mostraría la comprobación en rojo. A diferencia de GitLab, el detalle de las pruebas no se publica de forma nativa en el PR: habría que consultar el registro del *step* o enviar los resultados mediante Code Insights.

Evidencia 2 — Corrección y pruebas en verde

```
=== BITBUCKET :: step Pruebas unitarias (tras la correccion) ===
................
----------------------------------------------------------------------
Ran 16 tests in 0.001s

OK
```

Evidencia 3 — Despliegue

```
=== BITBUCKET :: step Desplegar en Produccion ===
== Despliegue de NotasApp ==
Entorno .....: production
Commit ......: efc1b9d2
Rama ........: main
Fecha .......: 2026-10-24 10:12:54
Resultado ...: despliegue en production completado.
```

El *step* se declara dentro de `pipelines.branches.main` y usa `trigger: manual`. Bitbucket registra el despliegue en el entorno, pero no define una URL ni conserva el historial de reversión que GitLab asocia al entorno.

Evidencia 4 — Historial y etiqueta

```
=== BITBUCKET :: historial final ===
*   efc1b9d (HEAD -> main) Merged in fix/script-despliegue-portable (pull request #2)
|\
| * c7dd0fc (fix/script-despliegue-portable) fix(ci): migra el despliegue a un script portable en Python
|/
*   b623000 (tag: v1.0.0) Merged in feature/promedio-ponderado (pull request #1)
|\
| * 4837826 (feature/promedio-ponderado) chore: excluye los artefactos de Python del control de versiones
| * d146a69 fix: valida la suma de pesos y redondea el promedio ponderado
| * bbe9c6f feat: agrega el calculo de promedio ponderado
|/
* 648f174 feat: version inicial de NotasApp con validacion, promedio y aprobacion

=== BITBUCKET :: estado del repositorio ===
## main
=== BITBUCKET :: pruebas sobre main ===
Ran 16 tests in 0.001s

OK

=== BITBUCKET :: etiquetas ===
v1.0.0 -> Release 1.0.0
```

La convención de mensajes de Bitbucket («Merged in … (pull request #n)») identifica claramente cada integración, igual que en GitLab lo hace «Merge branch … into 'main'».

Evidencia 5 — Publicación

```
=== BITBUCKET :: git push -u origin --all ===
branch 'feature/promedio-ponderado' set up to track 'origin/feature/promedio-ponderado'.
branch 'fix/script-despliegue-portable' set up to track 'origin/fix/script-despliegue-portable'.
branch 'main' set up to track 'origin/main'.
 * [new branch]      feature/promedio-ponderado -> feature/promedio-ponderado
 * [new branch]      fix/script-despliegue-portable -> fix/script-despliegue-portable
 * [new branch]      main -> main
=== BITBUCKET :: git push origin --tags ===
 * [new tag]         v1.0.0 -> v1.0.0
```

Sobre un servidor Bitbucket real, este *push* funcionaría con clave SSH o con un API token (las contraseñas de aplicación ya no se admiten desde julio de 2026).

7.6 Análisis del ejemplo de Bitbucket

Lo que el ejemplo demuestra a favor

- **Simplicidad:** tres claves de nivel superior (`image`, `definitions`, `pipelines`) frente a las nueve de GitLab.
- **Organización declarativa por evento:** `default`, `pull-requests` y `branches` expresan la política sin condicionales.
- **Reutilización con anclas YAML:** los tres *steps* se definen una vez y se referencian con `*build`, `*test`, `*estilo`, evitando duplicación.
- **Menos configuración para el mismo resultado:** 67 líneas frente a 76 con funcionalidad equivalente.
- **Despliegue por entorno sencillo:** la clave `deployment` y `trigger: manual` se leen sin conocer reglas de expresión.

Lo que el ejemplo evidencia en contra

- **Ejecución secuencial obligatoria:** no existe `needs`, por lo que el estilo esperaría aunque las pruebas pudieran correr antes; el pipeline completo tarda más.
- **Sin plantillas compartidas:** las anclas YAML solo funcionan dentro del mismo archivo; reutilizar configuración entre repositorios depende de Pipes o de copiar el archivo.
- **Menos control sobre el almacenamiento de resultados:** los *artefactos* no admiten reglas de conservación ni informes de prueba nativos.
- **Sin URL de entorno ni reversión nativa** asociada al despliegue.
- **Dependencia de la interfaz** para abrir el PR y, en planes inferiores, para que las comprobaciones de fusión sean efectivas.
- **50 minutos gratuitos al mes:** con este proyecto consumiría ~2 minutos por *push*, es decir, unos 25 *pushes* antes de tener que pagar.

8. Análisis comparativo de las dos demostraciones

Ambos proyectos partieron del mismo código y de las mismas pruebas. Todas las diferencias observadas provienen, por tanto, de la plataforma.

8.1 Resultados medidos

Métrica	GitLab	Bitbucket
Líneas del archivo de CI/CD	76	67
Claves de nivel superior	9	3
Unidades de ejecución	5 *jobs* en 3 etapas	3 *steps* reutilizables en 3 secciones
Paralelismo entre tareas de prueba	Sí (`needs` forma un grafo)	No (orden estrictamente secuencial)
Reutilización fuera del archivo	Sí (`include` , `extends`)	No (solo anclas dentro del archivo)
Condiciones de ejecución	Expresiones con `rules` y variables	Secciones por tipo de evento y rama
Entornos de despliegue	Nombre + URL + historial y reversión	Nombre del entorno, sin URL ni reversión
Informes de prueba en la revisión	Nativos (informes adjuntos al MR)	No nativos (Code Insights o revisar el registro)
Creación de la propuesta desde la consola	Sí (`git push -o merge_request.create`)	No
Resultado de la ejecución local	16 pruebas correctas, despliegue y etiqueta v1.0.0	16 pruebas correctas, despliegue y etiqueta v1.0.0

Interpretación: para un proyecto pequeño ambos son suficientes y el resultado funcional es idéntico. La diferencia se manifiesta al **crecer**: cuantos más módulos, entornos y equipos, más pesa la falta de paralelismo, de plantillas compartidas y de informes nativos en Bitbucket; y cuanto más pequeño y presupuestado sea el equipo, más pesa el coste y la complejidad de GitLab.

8.2 Traducción de conceptos entre las dos plataformas

Concepto	GitLab	Bitbucket
Propuesta de cambio	Merge Request	Pull Request
Unidad de trabajo del pipeline	*job* con `stage`	*step* dentro de una lista
Orden de fases	`stages` (lista ordenada)	Orden de aparición en la lista de *steps*
Imagen de ejecución	`image` global o por *job*	`image` global o por *step*
Caché	`cache` con clave derivada de archivos	`caches` con rutas declaradas
Artefactos	`artifacts` con rutas, informes y caducidad	`artifacts` como lista de rutas o comodines
Condicionales	`rules` con expresiones	Secciones `branches`, `pull-requests`, `tags`
Dependencias entre tareas	`needs` (grafo)	No existe; se hereda el orden
Reutilización	`include`, `extends`, plantillas	Anclas YAML y Pipes del mercado
Entorno	`environment` (nombre y URL)	`deployment` (nombre)
Ejecución manual	`when: manual`	`trigger: manual`
Autenticación por HTTPS	Personal Access Token	API token con permisos

9. Ventajas y desventajas de una plataforma sobre la otra

9.1 Dónde GitLab es claramente superior

Ventaja de GitLab	Por qué es relevante
CI/CD más potente	Grafo de dependencias, plantillas compartidas, *merge trains* y *pipelines* multicitados permiten pipelines completos en monorepos y microservicios, con mayor velocidad y menos duplicación.
Una sola plataforma	Evita integrar y pagar herramientas separadas de registro de imágenes, análisis de seguridad y planificación.
Seguridad integrada	El análisis estático, de dependencias, contenedores y secretos se ejecuta sin licencias de terceros (suite completa en Ultimate).
Autoalojamiento con paridad	Permite cumplir requisitos de soberanía de datos y trabajar en redes aisladas manteniendo las mismas funciones; solo la edición Community ya es gratuita y auditable.
Trazabilidad de despliegues	Entornos con URL, historial y reversión: facilita la operación y la auditoría de qué versión está publicada.
Informes nativos en la revisión	El resultado de las pruebas y de la cobertura aparece dentro del Merge Request, acortando el tiempo de diagnóstico.

9.2 Dónde Bitbucket es claramente superior

Ventaja de Bitbucket	Por qué es relevante
Coste	3,65 USD por usuario en Standard frente a 29 USD del Premium de GitLab: para 20 desarrolladores la diferencia supera los 6 000 USD al año.
Integración con Jira y Confluence	Enlaza confirmaciones, ramas, despliegues y tareas sin configuración adicional, lo que resulta decisivo en organizaciones ya asentadas en Atlassian.
Simplicidad	Menos conceptos, archivo de configuración más corto y curva de aprendizaje baja: un equipo nuevo produce pipelines útiles el primer día.
Nivel gratuito para equipos pequeños	Cinco usuarios con repositorios privados ilimitados, suficiente para proyectos académicos o de arranque.
Data Center con clúster activo-activo	Alta disponibilidad autoalojada consolidada, con soporte empresarial de Atlassian.
Seguridad de red asequible	Listas de IP permitidas y doble factor obligatorio en el plan Premium, cuyo precio sigue siendo inferior al de entrada de GitLab.

9.3 Resumen ejecutivo de la comparación

En una frase por plataforma

GitLab: máxima capacidad, control y autonomía (incluido el autoalojamiento), al precio de mayor complejidad y mayor coste por usuario.
Bitbucket: mínima fricción y menor coste, integrado de forma insuperable con Jira, a cambio de un CI/CD más limitado, ausencia de seguridad y registros nativos y dependencia del ecosistema Atlassian, que en 2026 se redujo aún más con el retiro de las incidencias y las *wikis* propias.

10. Conclusiones y recomendaciones

10.1 Conclusiones

1. **Las dos plataformas resuelven el problema central** del versionamiento en la nube: alojan repositorios Git, gestionan permisos, permiten revisar código y automatizan pruebas y despliegues. Con el mismo proyecto se obtuvo el mismo resultado funcional: 16 pruebas correctas, versión etiquetada y despliegue controlado.
2. **Las diferencias aparecen en el gobierno del proceso, no en el resultado.** GitLab ofreció paralelismo entre tareas, condiciones expresivas, entornos con reversión e informes nativos; Bitbucket resolvió lo mismo con menos líneas y menos conceptos, pero de forma secuencial y con menos control.
3. **El CI/CD de GitLab es más potente y el de Bitbucket más sencillo.** La evidencia es medible: 5 *jobs* con grafo de dependencias frente a 3 *steps* secuenciales, y 9 claves de configuración frente a 3.
4. **El coste es la diferencia más contundente.** El plan de entrada de pago de GitLab cuesta unas ocho veces más por usuario que el de Bitbucket, aunque incluye registro de contenedores, seguridad y planificación que en Bitbucket habría que contratar aparte.
5. **La integración con Jira es el argumento decisivo de Bitbucket.** Si la organización ya gestiona su trabajo en Jira y su documentación en Confluence, la ventaja de Bitbucket es difícil de igualar; si no, su valor disminuye notablemente, sobre todo tras el retiro de sus incidencias y *wikis* nativas en 2026.
6. **La seguridad y los registros nativos son exclusivos de GitLab** entre las dos opciones analizadas; en Bitbucket se depende de herramientas de terceros, lo que añade coste y complejidad de integración.
7. **El autoalojamiento es el terreno donde GitLab no tiene competencia** dentro de esta comparación: la Community Edition, bajo licencia MIT, permite operar una plataforma completa sin coste de licencia, algo imposible con Bitbucket.
8. **La autenticación obliga a actualizar automatizaciones.** Con la desaparición de las contraseñas de aplicación de Bitbucket (julio de 2026), cualquier script que usara HTTPS debe migrar a API tokens; en GitLab el uso de Personal Access Tokens o claves SSH ya era la práctica habitual.

10.2 Recomendaciones

Si su situación es…	Plataforma recomendada y razón
Ya usa Jira y Confluence y el presupuesto es limitado	**Bitbucket.** La integración es inmediata y el coste por usuario es el más bajo; complete con Pipes lo que no es nativo.
Equipo pequeño (≤ 5) con proyecto privado sencillo	**Bitbucket Free** si el uso intensivo de CI/CD es esporádico; **GitLab Free** si necesita más minutos de cómputo (400 frente a 50) o publicar sitios estáticos.
Necesita integración y entrega continua avanzadas, varios entornos y equipos numerosos	**GitLab.** El grafo de dependencias, las plantillas compartidas y los entornos con reversión reducen tiempos y errores de publicación.
Debe autoalojar por normativa o soberanía de datos	**GitLab Self-Managed** (Community Edition gratuita o Enterprise), por su paridad funcional y su capacidad de operar en red aislada.
Requiere análisis de seguridad integrado y cumplimiento	**GitLab** (la suite completa, en Ultimate), para evitar licencias y procesos separados.
Proyecto académico o de aprendizaje con énfasis en CI/CD	**GitLab**, por su nivel gratuito más generoso en minutos de cómputo y por el valor formativo de su pipeline declarativo completo.

10.3 Recomendación metodológica final

Antes de decidir, se recomienda ejecutar una prueba de concepto con el proyecto real durante un ciclo de desarrollo completo: medir los minutos de CI/CD consumidos, el número de integraciones necesarias y el coste total para el tamaño de equipo definitivo. En esta comparación, la verificación de precios resultó indispensable: durante la investigación se encontraron cifras distintas entre las propias páginas de Atlassian, lo que confirma que los valores deben confirmarse antes de cualquier decisión presupuestal.

11. Referencias y fuentes consultadas

Consulta realizada el 24 de octubre de 2026. Se priorizaron las fuentes oficiales de cada proveedor; los análisis de terceros se emplearon para datos de mercado y comparaciones de precio.

Documentación y páginas oficiales de GitLab

- GitLab — Planes y precios: `https://about.gitlab.com/pricing/`
- GitLab — Arquitectura y componentes: `https://docs.gitlab.com/development/architecture/`
- GitLab — Gitaly y Gitaly Cluster: `https://docs.gitlab.com/administration/gitaly/`
- GitLab — GitLab Runner y sus ejecutores: `https://docs.gitlab.com/runner/` y `https://docs.gitlab.com/runner/executors/`
- GitLab — Referencia de sintaxis de CI/CD en YAML: `https://docs.gitlab.com/ci/yaml/`
- GitLab — Ramas protegidas: `https://docs.gitlab.com/user/project/repository/branches/protected/`
- GitLab — Entornos y despliegues: `https://docs.gitlab.com/ci/environments/`
- GitLab — Container Registry: `https://docs.gitlab.com/user/packages/container_registry/`
- GitLab — Tokens de acceso personal: `https://docs.gitlab.com/user/profile/personal_access_tokens/`
- GitLab Handbook — Historia de la compañía: `https://handbook.gitlab.com/handbook/company/history/`
- GitLab — Requisitos de instalación (memoria y hardware): `https://gitlab.com/gitlab-org/gitlab/-/blob/master/doc/install/requirements.md`

Documentación y páginas oficiales de Bitbucket / Atlassian

- Bitbucket — Precios: `https://bitbucket.org/product/pricing`
- Atlassian — Precios de Bitbucket: `https://www.atlassian.com/software/bitbucket/pricing`
- Bitbucket — Configuración de `bitbucket-pipelines.yml`: `https://support.atlassian.com/bitbucket-cloud/docs/configure-bitbucket-pipelinesyml/`
- Bitbucket — Descripción general (Cloud frente a Data Center): `https://bitbucket.org/product/guides/getting-started/overview`
- Atlassian — Comparación Cloud frente a Data Center: `https://www.atlassian.com/migration/assess/compare-cloud-data-center/bitbucket`
- Bitbucket — Registro de cambios (retiro de contraseñas de aplicación): `https://developer.atlassian.com/cloud/bitbucket/changelog/`
- Atlassian — Guía para migrar incidencias de Bitbucket a Jira: `https://support.atlassian.com/bitbucket-cloud/kb/guide-to-migrating-issues-from-bitbucket-cloud-to-jira-cloud/`
- Atlassian — Hoja de ruta de Data Center: `https://www.atlassian.com/roadmap/data-center`

Análisis de terceros y datos de mercado (2026)

- devops-daily — Comparativa Bitbucket frente a GitLab: `https://devops-daily.com/comparisons/bitbucket-vs-gitlab`
- SaaSCompared — GitHub, GitLab y Bitbucket en 2026: `https://saascompared.com/blog/github-vs-gitlab-vs-bitbucket-2026-refresh`
- FOSS Post — Estadísticas de cuota de mercado de Git: `https://fosspost.org/git-market-share-statistics/`
- JetBrains / AWSQuality — Herramientas de CI/CD más usadas: `https://www.awsquality.com/best-ci-cd-tools-what-the-data-actually-shows/`

- eesel — Precios de Bitbucket y de GitLab: `https://www.eesel.ai/blog/bitbucket-pricing` y `https://www.eesel.ai/blog/gitlab-pricing`

Advertencia sobre los datos

- Los precios varían por región, moneda, impuestos y tamaño de equipo, y cambian con frecuencia. Deben verificarse en la web oficial antes de presupuestar.
- En GitLab, el plan Ultimate se ofrece con precio personalizado; el nivel gratuito limita a 5 usuarios por grupo de nivel superior en la nube, límite que no aplica en instalaciones autoalojadas.
- Las cuotas de mercado son estimaciones de analistas y varían según la metodología (repositorios públicos o privados, empresas o desarrolladores).

12. Anexos: archivos entregados y reproducción

12.1 Archivos entregados

Archivo	Contenido
Informe-GitLab-vs-Bitbucket.pdf	Este informe (investigación, demostraciones y análisis).
demos/gitlab/	Repositorio Git del Ejemplo 1: `.gitlab-ci.yml` , `src/notas.py` , `tests/test_notas.py` , `scripts/deploy.py` , `README.md` , `.gitignore` e historial completo con *commits* y etiqueta `v1.0.0` .
demos/bitbucket/	Repositorio Git del Ejemplo 2: `bitbucket-pipelines.yml` y los mismos archivos de aplicación, con su propio historial y etiqueta `v1.0.0`.

12.2 Cómo reproducir la demostración

```
# Ejecutar las pruebas unitarias de cualquiera de los dos proyectos
cd demos/gitlab          # o demos/bitbucket
python -m unittest discover -s tests -v      # 16 pruebas

# Simular el paso de despliegue de los pipelines
python scripts/deploy.py produccion

# Consultar el historial y las etiquetas de la demostración
git log --graph --oneline --decorate --all
git tag -l

# Validar la sintaxis del archivo de configuración de CI/CD
python -c "import yaml;print(list(yaml.safe_load(open('.gitlab-ci.yml')).keys()))"

# Publicar en un remoto (en la plataforma real, previa configuración de la URL y las credenciales)
git remote add origin
git push -u origin --all
git push origin --tags
```

12.3 Correspondencia entre la demostración y la plataforma real

En la demostración	En GitLab o Bitbucket
`git merge --no-ff` con mensaje de integración	Botón *Merge* del Merge Request / Pull Request, tras las aprobaciones y con el pipeline correcto.
Ejecución manual de `python -m unittest`	*Job* `test:unitarias` (GitLab) o *step* «Pruebas unitarias» (Bitbucket) dentro del pipeline.
`python scripts/deploy.py produccion`	*Job* `deploy:produccion` con `when: manual` (GitLab) o *step* con `trigger: manual` (Bitbucket).
Repositorio desnudo local como remoto	Repositorio alojado en GitLab.com o Bitbucket.org, con autenticación por SSH o token.
Etiqueta `v1.0.0`	Etiqueta del proyecto y, en GitLab, posibilidad de crear una «Release» formal.

Las marcas GitLab y Bitbucket pertenecen a sus respectivos titulares.